

DEGAiN TRAINING

Bacterial Genome-Wide Association Studies in Diarrhoeagenic *E. coli*

Overview

This workbook guides participants through the complete workflow on uncovering the genetic variants associated with bacterial phenotypes. We also highlight methods on correcting for confounding factors that can influence the accuracy and reliability of bacterial GWAS results.

Learning objectives

On completing this practical session, participants will be able to:

- Prepare relevant phenotype data.
- Generate genetic variant types such as unitigs and kmers, using **unitig-caller** and **fsm-lite**, respectively.
- Perform bacterial GWAS using **pyseer**.
- Annotate significant hits.
- Generate relevant plots using scripts, web-based tools (Phandango) and/or R.

Note: There is currently no standardized protocol for bacterial genome-wide association studies (BGWAS), especially regarding the number of strains required, types of genetic variants considered, and choice of analytical tools.

No bioinformatics tool is universally considered "most correct." Every tool has its own strengths and limitations. The tools presented in this workbook have been widely published, validated, and used in research and public health applications. Users should understand each tool's assumptions and limitations when interpreting results.

Open a tmux session before you start

```
tmux new -s mysession
```

Other useful tmux command

```
# List available session
tmux ls

# Attach to a named session
tmux attach -t mysession

# Leave a session without stopping it
ctrl-b d
```

Your expected project directory layout on the server

```
# within the directory for each group
gwas/
├─ clusterX/
│   ├── bakta/           # genome annotations
│   ├── fasta/           # INPUT: assembled genomes
│   ├── gff3/            # INPUT: genome annotations in gff3 format
│   ├── panaroo_output/  # pangenome output
│   └─ bakta/            # reference annotation (incomplete)
```

One of the motivations for using unitigs instead of raw k-mers is that unitigs collapse long stretches of identical k-mer patterns into a single feature. Because unitigs remove much of the redundancy before testing, they greatly reduce the number of association tests, improving computational efficiency and statistical power. This is one reason why modern bacterial GWAS pipelines increasingly favor unitigs over raw k-mers.

GWAS using Pyseer (Demo)

```
# SNP and COG association with fixed effect model
## estimate population structure
mash sketch -s 10000 -o mash_sketch assemblies/*.fa
mash dist Data/mash_sketch.msh Data/mash_sketch.msh | square_mash > Data/mash.tsv

## perform multidimensionality scaling (MDS) to correct for population structure. Look at the
screed plot
screed_plot_pyseer mash.tsv

## run pyseer using cluster of orthologous genes (COGs)
pyseer --phenotypes Data/resistances.pheno --pres Data/gene_presence_absence.Rtab --distances
Data/mash.tsv --save-m mash_mds --max-dimensions 8 > Data/penicillin_COGs.txt

## run pyseer using SNPs. Tests for lineage effects is also turned on here (--lineage)
pyseer --phenotypes Data/resistances.pheno --vcf Data/snps.vcf.gz --load-m Results/mash_mds.pkl
--lineage --print-samples > Results/penicillin_SNPs.txt

## convert coordinates and p-values to .plot
cat <(echo "#CHR SNP BP minLOG10(P) log10(p) r^2") <(paste <(sed '1d' Results/
penicillin_SNPs.txt | cut -d "_" -f 2) <(sed '1d' Results/ penicillin_SNPs.txt | cut -f 4) |
awk '{p = -log($2)/log(10); print "26",".", $1,p,p,"0"}' ) | tr ' ' '\t' > Results/
penicillin_SNPs.plot

## drag and drop reference annotation file (23FSpn.gff) and the generated plot
(penicillin_SNPs.plot) in phandango
```

GWAS using Pyseer (Demo)]

```
# kmer association with mixed effect model
## generate and count kmers
fsm-lite -l ../fsm_file_list.txt -s 6 -S 610 -v -t fsm_kmers | gzip -c - >
../Results/fsm_kmers.txt.gz

## generate kinship matrix. Needed to control for population structure.
python3 scripts/phylogeny_distance.py --lmm Data/core_genome_aln.tree > Results/phylogeny_K.tsv

## run pyseer with linear mixed effect model (--lmm), using the kinship matrix generated
pyseer --lmm --phenotypes Data/resistances.pheno --kmers Results/fsm_kmers.txt.gz --similarity
Results/phylogeny_K.tsv --output-patterns Results/kmer_patterns.txt --cpu 8 >
Results/penicillin_kmers.txt

## determine a significance threshold using the number of unique k-mer patterns
python3 BGWAS/scripts/count_patterns.py Results/penicillin_kmers.txt

## We can also create a Q-Q plot to check that p-values are not inflated.
python3 BGWAS/scripts/qq_plot.py Results/penicillin_kmers.txt

## Work only with kmers which exceeded the significance threshold in the mixed model analysis.
## Or use
## awk -v OFS='\t' 'NR==1{print $0};$4<3.53E-09{print $0}' Results/penicillin_kmers.txt >
Results/significant_kmers.txt
cat <(head -1 Results/penicillin_kmers.txt) <(awk '$4<3.53E-09 {print $0}'
Results/penicillin_kmers.txt) > Results/significant_kmers.txt

## then map to a single reference. It is a good idea to map to range of references to help with
an interpretation for all of the significant k-mers.
phandango_mapper Results/significant_kmers.txt Data/Spn23F.fa Results/Spn23F_kmers.plot
# Visualize plot in phandango

## Annotating k-mers. We'll start by creating a references.txt file listing the annotations we
wish to use
annotate_hits_pyseer ../Results/significant_kmers.txt references.txt
../Results/annotated_kmers.txt

## Finally, we can summarise these annotations to create a plot of significant genes.
python3 scripts/summarise_annotations.py Results/annotated_kmers.txt > Results/gene_hits.txt
## plot results in R (see example code in the "scripts" folder.
```

GWAS using Pyseer (Demo - Unitig association)

```
# Gene variant type: unitigs
cd gwas/cluster
ls -d -1 BGWAS/Data/assemblies/*.fa > filepath_for_unitig-caller.txt

# Call unitigs. Note: ref refers to path to assemblies
unitig-caller --call --ref filepath_for_unitig-caller.txt --pyseer --out unitigs
gzip -c unitigs.pyseer > unitigs.pyseer.gz

# following the generation of unitigs.pyseer.gz file, repeat the same steps afterwards like in
the previous GWAS Demo where we used kmers as genetic variant type.
#####
```

Discussion questions

What differences do you observe when kmer was used as a genetic variant versus when unitig was used. How do the GWAS results differ when k-mers are used instead of unitigs as the genetic variants? What might explain these differences?

GWAS using Pyseer (Practical Session)

```
# =====
# PYSEER TUTORIAL - PRACTICAL SESSION
# =====

# Assumption:
# The core genome alignment (core_gene_alignment_filtered.aln)
# has already been generated using Panaroo.

# Before running pyseer, prepare the following:
# 1. Phenotype file (binary or continuous)
# 2. Unitigs (unitigs.pyseer.gz). See step 1.
# 3. Midpoint-rooted phylogenetic tree. See step 3.
# 4. Similarity (kinship) matrix. See step 4.

# Move to the working directory
cd ~/groupX_clusterXX/gwas/clusterXX

# -----
# Step 1. Generate unitigs
# -----

# Create a file containing the paths to all genome assemblies
ls -d -1 fasta/*.fasta > filepath_for_unitig-caller.txt

# Generate unitigs.
# (--ref specifies the file containing the assembly paths.)
unitig-caller --call --ref filepath_for_unitig-caller.txt --pyseer --out unitigs

# Compress the output for pyseer
gzip -c unitigs.pyseer > unitigs.pyseer.gz
```

```

# -----
# Step 2. Build a phylogenetic tree
# -----

# FastTree is used here because it is fast.
# Other tree-building programs (e.g. IQ-TREE or RAXML) can also be used.

FastTree -nt panaroo_output/core_gene_alignment_filtered.aln > fasttree_tree.tree

# -----
# Step 3. Midpoint-root the tree
# -----

Rscript scripts/midpoint_root_tree.R fasttree_tree.tree

# -----
# Step 4. Generate the similarity (kinship) matrix
# -----
# The similarity matrix is required for the Linear Mixed Model (LMM)
# to account for population structure.

python scripts/phylogeny_distance.py --lmm fasttree_tree.tree_midpoint.tree > phylogeny_K.tsv

# -----
# Step 5. Run pyseer
# -----

# Perform GWAS using a Linear Mixed Model (LMM).

Pyseer --lmm --phenotypes clusterX_case_control_phenotypev2.txt --kmers unitigs.pyseer.gz --
similarity phylogeny_K.tsv --output-patterns unitigs_patterns.txt --cpu 8 >
case_control_unitigs.txt

```

```

# -----
# Step 6. Determine the significance threshold
# -----

# Count the number of unique presence/absence patterns to calculate
# the multiple-testing threshold.

python scripts/count_patterns.py unitigs_patterns.txt

# -----
# Step 7. Select significant unitigs
# -----
# Keep the header and all unitigs with an LRT p-value below # the significance threshold. # #
# In this output, column 4 contains the LRT p-value.

awk -v OFS='\t' 'NR == 1 {print; next} $4 < 2.82E-08 {print}' case_control_unitigs.txt >
significant_unitigs.txt

# -----
# Step 8. Identify suitable reference genomes
# -----
# Find a closely related reference genome for mapping and annotation.
# Mapping against several suitable references may improve interpretation
# because some significant unitigs may be absent from a single reference.

bash scripts/find_top5_closest_references.sh

# -----
# Step 9. Map significant unitigs to a reference genome
# -----
# Map the significant unitigs to each selected reference genome (one at a time) to determine
# their genomic location. If a unitig does not map to one reference, repeat the mapping with
# another closely related reference identified in Step 8
phandango_mapper significant_unitigs.txt reference1.fna reference1.plot
phandango_mapper significant_unitigs.txt reference2.fna reference2.plot

```

```
# -----  
# Step 10. Annotate significant unitigs  
# -----  
# references.txt should contain the reference genomes that will be used  
# to identify the genes or genomic regions containing significant unitigs.  
  
annotate_hits_pyseer significant_unitigs.txt references.txt annotated_unitigs.txt  
  
# -----  
# Step 11. Plot  
# -----  
# Finally, we can summarise these annotations to create a plot of significant variants.  
  
python3 scripts/summarise_annotations.py Results/annotated_unitigs.txt > Results/gene_hits.txt  
## plot results in R
```